



ARTIFICIAL LIFE

PYTHON SIMBOTS

A-LIFE

- A-Life is a field of study wherein researchers examine systems related to **natural life**, **its processes**, and **its evolution**, *through the use of simulations with computer models, robotics, and biochemistry.*

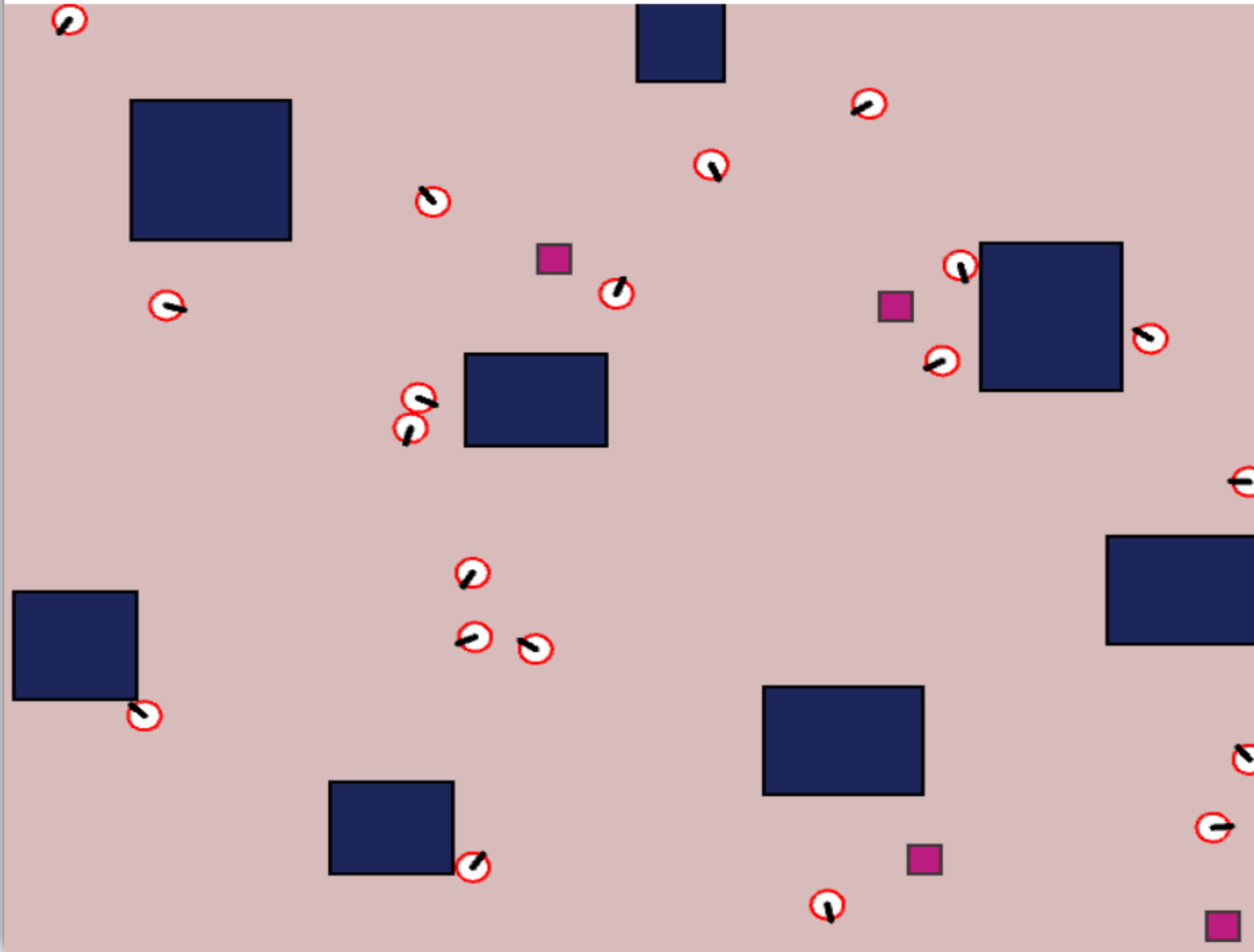
- from https://en.wikipedia.org/wiki/Artificial_life

AFTER YOU HAVE LEARNED GA, USED FOR CREATING FUZZY RULES FOR YOU

- This assignment is about **to extend the algorithm a little bit.**
- Since GA emulates the process of evolution
 - considered as a natural process of many living things.
- Instead of looking the process by fixed time of many generations,
 - this time, all living things will be live their lives until they die.
- One who lives longer
 - have more chances to reproduce or spread some of its DNA into the population.
- Once there is someone to die,
 - the nature is called upon to create a new individual by mating a couple from the remaining population.
- When the time goes on,
 - the improved ones survive and live their lives,
 - finally become the dominated ones.

TO CREATE AN ARTIFICIAL LIFE, THESE ARE THE STEPS ..

- At start, **a population of 20 robot individuals** is generated. As usual, their chromosomes are randomly formed. To stay alive, all robots must have their energy (**they were born with 300**).
 - The energy is not only decreased every time, but also consumed more when the robot is not in its well status.
 - For example, if the robot hits something, the energy is spent faster. Other usual statuses are lazy, headache, and etc.
- In the environment, all robots are put into the test. (**The energy is the key**)
 - To live longer, the robots have to preserve their energy and to find food getting more additional energy to life.
 - Food are limited, but not empty. If one food was taken, another food is formed in another place randomly located.
 - Each **robot is able to see others**. Therefore, other robots are considered as moving obstacles. There are also static obstacles, which are not moving at all.
- When one robot dies, the process to generate the new robot is taking place for replacing the dead.
 - A couple of robots are randomly picked from the remaining ones in the population.
 - Then, the crossover is called to reproduce a new robot from its parents.
 - In addition, mutation is also applied rarely.
- The simulation time is counting continuously until reaching the maximum tick (100000).
- Learning curve is plotted from **amount of dead robots** in **different time intervals**.
- At the end, **the robot with highest energy** is selected to be **the final answer**.



Simulation Count: 1
Time: 46
Eat: 0
Food Moved: 0
Score: 0 %

[Shortcuts]
n: to change the food
position

```
277 if __name__ == '__main__':
278
279     app = PySimbotApp(robot_cls=StupidRobot,
280                       num_robots=20,
281                       num_objectives=4,
282                       theme='default',
283                       simulation_forever=False,
284                       max_tick=100000,
285                       interval=1/1000.0,
286                       food_move_after_eat=True,
287                       robot_see_each_other=True,
288                       # map="no_wall",
289                       customfn_before_simulation=before_simulation,
290                       customfn_after_simulation=after_simulation)
291     app.run()
```

```
92     if int(answerMove) == 0 and int(answerTurn) == 0:
93         self.lazy_count += 1
94
95     if int(answerMove) < 0 or abs(answerTurn) > 30:
96         self.headache_count += 1
97
98     self.turn(answerTurn)
99     self.move(answerMove)
100
101     # every step the robot lost its energy
102     self.energy -= 1
103
104     # if the robot hit, it also lost energy
105     if self.just_hit :
106         # self.energy -= Y
107         pass
108
109     # if the robot eat food, it gets some energy back
110     if self.just_eat :
111         # self.energy += X
112         pass
113
114     if self.energy < 0 :
115         # die and find a new robot
116         print (self._sm.iteration)
117
118         # record that there is a dead one
119
120         # generate new robot by usubg genetic operations
121         temp = StupidRobot()
122         temp = self.generate_new_robot()
123         self.RULES = deepcopy(temp.RULES)
124
125         # give this new born some energy
126         self.energy += 300
127         pass
```

```
129 def generate_new_robot(self):
130     simbot = self._sm
131     num_robots = len(simbot.robots)
132
133     def select() -> StupidRobot:
134         index = random.randrange(num_robots)
135         return simbot.robots[index]
136
137     select1 = select()    # design the way for selection by yourself
138     select2 = select()    # design the way for selection by yourself
139
140     while select1 == select2:
141         select2 = select()
142
143     temp = StupidRobot()
144
145     # Doing crossover
146     #     using next_gen_robots for temporary keep the offsprings, later they will be copy
147     #     to the robots
148
149     # Doing mutation
150     #     generally scan for all next_gen_robots we have created, and with very low
151     #     propability, change one byte to a new random value.
152
153     # Making a new random robot
154
155     return temp
```